jenkins_robotframework 完整部署手册

适用环境：Debian 13 Master + Jenkins Agent + Windows 浏览器访问。
当前服务器代码目录固定为：`/opt/jenkins_robotframework`。
最终外部入口统一由 Nginx + HTTPS 收口，不直接暴露 `platform-api`、Vite dev server 或 Jenkins 原始端口。

日常重启服务、切换虚拟环境、查看日志和快速验证命令见 `deploy/SERVER_OPERATIONS.md`。

1. 最终访问形态

推荐最终访问路径：

```
https://10.71.210.104/          -> automation-portal 前端页面
https://10.71.210.104/api/      -> platform-api FastAPI
https://10.71.210.104/jenkins/  -> Jenkins
```

如果后续有正式域名，把 `10.71.210.104` 替换为域名，例如：

```
https://jenkins.company.com/
https://jenkins.company.com/api/
https://jenkins.company.com/jenkins/
```

内部监听建议：

```
127.0.0.1:8080  Jenkins，带 /jenkins prefix
127.0.0.1:8000  platform-api
无外部端口      automation-portal，只部署 dist 静态文件
```

2. 模块与部署目录

仓库目录：

```
/opt/jenkins_robotframework
├── automation-portal/        React/Vite 前端，最终部署 dist/
├── platform-api/             FastAPI 后端，systemd 常驻
├── jenkins-integration/      Jenkinsfile、JCasC、Job DSL、桥接脚本
├── test-workflow-runner/     Robot workflow runner、internal_tools worker
├── deploy/                   部署文档、env 示例、后续 systemd/nginx 模板
└── docs/                     架构与 step 文档
```

运行期目录：

```
/var/lib/test-workflow-runner/
├── data/
├── logs/
└── backups/

/var/lib/test-workflow-runner/internal-tools/
├── artifacts/
└── logs/
```

3. 总体部署顺序

建议按下面顺序做：

1. 准备 Master 基础依赖与 `/opt/jenkins_robotframework`。
2. 部署 Jenkins Master，设置 `/jenkins` prefix。
3. 配置 Nginx HTTPS，先代理 Jenkins。
4. 配置 Jenkins Agent。
5. 部署 `platform-api` systemd 服务。
6. 构建并部署 `automation-portal/dist`。
7. 扩展 Nginx：`/`、`/api/`、`/jenkins/` 三路统一收口。
8. 配置 Jenkins job：`robot/robot-execution`。
9. 可选：部署 standalone `internal_tools.worker` systemd 服务。
10. 从 Windows 浏览器做端到端 smoke。

4. Master 基础准备

在 Master 服务器 `10.71.210.104` 上执行。

4.1 安装基础包

```
sudo apt update
sudo apt upgrade -y

sudo apt install -y \
  python3 \
  python3-pip \
  python3-venv \
  openjdk-17-jre \
  git \
  curl \
  vim \
  nginx \
  openssl
```

验证：

```
python3 --version
java -version
git --version
nginx -v
```

4.2 创建部署目录

```
sudo mkdir -p /opt/jenkins_robotframework
sudo mkdir -p /var/lib/test-workflow-runner/{logs,backups,data}
sudo mkdir -p /var/lib/test-workflow-runner/internal-tools/{artifacts,logs}

sudo chown -R ute:ute /opt/jenkins_robotframework
sudo chown -R ute:ute /var/lib/test-workflow-runner
```

4.3 同步代码

首次部署：

```
cd /opt
sudo git clone https://github.com/stella555359/jenkins_robotframework.git
sudo chown -R ute:ute /opt/jenkins_robotframework
```

后续更新：

```
cd /opt/jenkins_robotframework
git pull
```

服务器只做部署与运行配置，不在服务器上长期手改业务源码。

5. Jenkins Master 部署

5.1 安装 Jenkins

```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io.key

echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install -y jenkins
```

5.2 启动 Jenkins

```
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins --no-pager
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

5.3 设置 Jenkins Prefix

因为最终通过 `/jenkins/` 暴露，必须设置 prefix。

```
sudo systemctl edit jenkins
```

写入：

```
[Service]
Environment="JENKINS_PREFIX=/jenkins"
```

保存后：

```
sudo systemctl daemon-reload
sudo systemctl restart jenkins
sudo systemctl status jenkins --no-pager
```

本机验证：

```
curl -I http://127.0.0.1:8080/jenkins/
```

5.4 Jenkins 插件

建议安装：

```
Pipeline
Git
Credentials Binding
SSH Build Agents
Configuration as Code
Job DSL
Timestamper
AnsiColor
```

```
HTML Publisher
JUnit
```

5.5 Jenkins URL

Nginx HTTPS 配好后，在 Jenkins 页面设置：

```
Manage Jenkins -> System -> Jenkins URL
```

值：

```
https://10.71.210.104/jenkins/
```

如果使用正式域名：

```
https://jenkins.company.com/jenkins/
```

6. Nginx 与 HTTPS

6.1 证书

如果还没有正式证书，可以先用自签名证书：

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:4096 \
  -keyout /etc/ssl/private/jenkins-kpi-platform.key \
  -out /etc/ssl/certs/jenkins-kpi-platform.crt \
  -subj "/CN=10.71.210.104"
```

证书文件位置：

```
/etc/ssl/certs/jenkins-kpi-platform.crt
/etc/ssl/private/jenkins-kpi-platform.key
```

6.2 Nginx 配置文件位置

仓库建议维护位置：

```
/opt/jenkins_robotframework/deploy/nginx/jenkins-kpi-platform.conf
```

系统启用位置：

```
/etc/nginx/sites-available/jenkins-kpi-platform.conf
/etc/nginx/sites-enabled/jenkins-kpi-platform.conf
```

如果服务器上已经有旧配置，例如：

```
/etc/nginx/sites-enabled/jenkins-kpi-platform.conf
```

不要先删 Jenkins 现网入口，优先确认它是否是符号链接：

```
sudo ls -l /etc/nginx/sites-enabled/jenkins-kpi-platform.conf
```

常见情况有两种：

1. 如果它链接到 `/etc/nginx/sites-available/jenkins-kpi-platform.conf`，优先改 `sites-available` 里的源文件。
2. 如果它就是一个实际文件，也可以直接原地修改这个文件，不必强制改名成 `jenkins-kpi-platform.conf`。

也就是说，这一节里的 `jenkins-kpi-platform.conf` 是推荐文件名，不是必须文件名。服务器已经稳定运行时，保留旧文件名、只更新内容，风险更低。

当前仓库如果还没有 `deploy/nginx/` 目录，可以在服务器部署时创建：

```
mkdir -p /opt/jenkins_robotframework/deploy/nginx
```

6.2.1 从已有 `jenkins-kpi-platform.conf` 迁移

如果当前线上配置是：

```
location /jenkins/ {
  proxy_pass http://127.0.0.1:8080/jenkins/;
}

location /reports/ {
  proxy_pass http://127.0.0.1:8000/;
}

location /kpi/ {
  proxy_pass http://127.0.0.1:8001/;
}
```

而当前项目目标是本文档里的统一入口：

```
/jenkins/ -> Jenkins
/api/     -> platform-api
/         -> automation-portal dist
```

则迁移原则是：

1. 保留 `/jenkins/` 不变。
2. 删除旧的 `/reports/` 反代。
3. 删除旧的 `/kpi/` 反代。
4. 新增 `/api/` 反代到 `127.0.0.1:8000/api/`。
5. 新增 `/` 指向 `automation-portal/dist` 静态页面。

如果证书已经在用，例如：

```
/etc/ssl/certs/jenkins-kpi-platform.crt
/etc/ssl/private/jenkins-kpi-platform.key
```

第一轮可以继续沿用，不需要为了对齐文档专门重签证书或改证书文件名。

6.3 完整 Nginx 配置

`/opt/jenkins_robotframework/deploy/nginx/jenkins-kpi-platform.conf`：

```
server {
    listen 80;
    server_name 10.71.210.104;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name 10.71.210.104;

    ssl_certificate /etc/ssl/certs/jenkins-kpi-platform.crt;
    ssl_certificate_key /etc/ssl/private/jenkins-kpi-platform.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers on;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    # Serve Jenkins archived artifacts (log.html etc.) without CSP restriction.
    # Jenkins default CSP blocks inline JavaScript, which breaks Robot Framework
```

```
log.html.
    # This block MUST appear before the general /jenkins/ prefix location.
    location ~ ^/jenkins/job/.+/artifact/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_hide_header Content-Security-Policy;
    }

    location /jenkins/ {
        proxy_pass http://127.0.0.1:8080/jenkins/;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8000/api/;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
    }

    location / {
        root /opt/jenkins_robotframework/automation-portal/dist;
        try_files $uri $uri/ /index.html;
    }
}
```

启用：

```
sudo cp /opt/jenkins_robotframework/deploy/nginx/jenkins-kpi-platform.conf
/etc/nginx/sites-available/jenkins-kpi-platform.conf
sudo ln -sf /etc/nginx/sites-available/jenkins-kpi-platform.conf /etc/nginx/sites-
enabled/jenkins-kpi-platform.conf
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

如果服务器已经在使用 `jenkins-kpi-platform.conf`，更稳妥的做法不是新建第二份站点文件，而是直接把旧文件内容改成下面目标配置，再执行：

```
sudo nginx -t
sudo systemctl reload nginx
```

建议迁移后的实际内容如下。若你沿用旧证书文件名，只需要把 `ssl_certificate` 和 `ssl_certificate_key` 两行替换成服务器当前已存在的证书路径即可：

```
server {
  listen 80;
  server_name 10.71.210.104;

  return 301 https://$host$request_uri;
}

server {
  listen 443 ssl;
  server_name 10.71.210.104;

  ssl_certificate /etc/ssl/certs/jenkins-kpi-platform.crt;
  ssl_certificate_key /etc/ssl/private/jenkins-kpi-platform.key;
  ssl_protocols TLSv1.2 TLSv1.3;
  ssl_prefer_server_ciphers on;

  proxy_set_header Host $host;
  proxy_set_header X-Real-IP $remote_addr;
  proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  proxy_set_header X-Forwarded-Proto $scheme;

  # Serve Jenkins archived artifacts (log.html etc.) without CSP restriction.
  location ~ ^/jenkins/job/.+/artifact/ {
    proxy_pass http://127.0.0.1:8080;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_hide_header Content-Security-Policy;
  }

  location /jenkins/ {
    proxy_pass http://127.0.0.1:8080/jenkins/;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
  }

  location /api/ {
    proxy_pass http://127.0.0.1:8000/api/;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
  }

  location / {
    root /opt/jenkins_robotframework/automation-portal/dist;
    try_files $uri $uri/ /index.html;
  }
}
```

验证：

```
curl -k -I https://127.0.0.1/jenkins/
curl -k -I https://127.0.0.1/api/health
curl -k -I https://127.0.0.1/
```

结果判读：

1. `/jenkins/` 只要返回里能看到 `X-Jenkins`、`X-Hudson` 或 Jenkins 的 `Set-Cookie`，即使状态码是 `403`，也说明 Nginx 到 Jenkins 的反代链路已经通了。`curl -I` 使用的是 `HEAD`，Jenkins 在未登录或开启权限控制时返回 `403` 是常见现象。
2. `/api/health` 返回 `502 Bad Gateway`，说明 Nginx 已经接收到请求，但它连不上 `127.0.0.1:8000` 的 `platform-api`。优先检查 `platform-api` 是否启动、是否监听 `127.0.0.1:8000`。
3. `/` 返回 `500 Internal Server Error`，通常不是 HTTPS 或反代本身有问题，而是 Portal 静态目录不存在、`index.html` 缺失，或 Nginx 对 `/opt/jenkins_robotframework/automation-portal/dist` 没有访问权限。

如果你看到的正是：

```
/jenkins/ -> 403
/api/health -> 502
/ -> 500
```

那可以解读为：

1. Jenkins 反代基本正常。
2. `platform-api` 还没就绪。
3. `automation-portal/dist` 还没就绪。

对应排查命令：

```
sudo systemctl status platform-api --no-pager
ss -ltnp | grep 8000
curl http://127.0.0.1:8000/api/health

ls -la /opt/jenkins_robotframework/automation-portal/dist
ls -la /opt/jenkins_robotframework/automation-portal/dist/index.html
sudo journalctl -u nginx -n 100 --no-pager
```

Windows 浏览器验证：

```
https://10.71.210.104/
https://10.71.210.104/api/health
https://10.71.210.104/jenkins/
```

7. Jenkins Agent 部署

以下在 Agent 服务器 `10.57.159.149` 上执行。

7.1 安装基础包

```
sudo apt update
sudo apt upgrade -y

sudo apt install -y \
  python3 \
  python3-pip \
  python3-venv \
  openjdk-17-jre \
  git \
  openssh-server \
  curl \
  vim
```

7.2 创建 Jenkins 执行用户和目录

```
sudo useradd -m -s /bin/bash jenkins || true
echo "jenkins ALL=(ALL) NOPASSWD:ALL" | sudo tee /etc/sudoers.d/jenkins

sudo mkdir -p /automation/{workspace,venv,logs,downloads}
sudo chown -R jenkins:jenkins /automation
```

验证：

```
java -version
readlink -f "$(which java)"
id jenkins
ls -ld /automation /automation/workspace
systemctl status ssh --no-pager || systemctl status sshd --no-pager
```

7.3 Master 到 Agent SSH

如果你忘了 `t813-agent` 登录 key、`robotws` checkout key、`testline_configuration` checkout key 分别怎么生成、如何配置到 JCasC、以及哪两把可以复用，统一参考 c:/TA/jenkins_robotframework/docs/overview/jenkins-ssh-key-setup.md。

在 Master 上生成 key：

```
ssh-keygen -t rsa -b 4096 -m PEM -C "jenkins-master" -f ~/.ssh/jenkins_agent_rsa -
N ""
```

这里建议显式带上 `-m PEM`，让 `t813-agent` 这把登录私钥保持为 `-----BEGIN RSA PRIVATE KEY-----` 格式，避免 Jenkins SSH launcher 报 `PEM problem: it is of unknown type`。

把公钥加入 Agent 的 `jenkins` 用户：

```
ssh-copy-id -i ~/.ssh/jenkins_agent_rsa.pub jenkins@10.57.159.149
```

验证：

```
ssh -i ~/.ssh/jenkins_agent_rsa jenkins@10.57.159.149 "echo 'SSH OK'"
```

7.4 Jenkins 页面添加 Agent

路径：

```
Manage Jenkins -> Nodes -> New Node
```

推荐配置：

```
Node name: t813-agent
Type: Permanent Agent
Remote root directory: /automation/workspace
Labels: t813 robot
Usage: Only build jobs with label expressions matching this node
Launch method: Launch agents via SSH
Host: 10.57.159.149
Credentials: t813-agent-ssh
Host Key Verification Strategy: Non verifying Verification Strategy（第一轮可用）
```

Credentials 新增：

```
Kind: SSH Username with private key
Scope: Global
Username: jenkins
Private Key: Enter directly，粘贴 Master 上 ~/.ssh/jenkins_agent_rsa
ID: t813-agent-ssh
Description: SSH key for t813-agent
```

8. Jenkins Integration 配置

相关仓库文件：

```
/opt/jenkins_robotframework/jenkins-integration/pipelines/robot-
execution.Jenkinsfile
/opt/jenkins_robotframework/jenkins-integration/scripts/
/opt/jenkins_robotframework/jenkins-integration/jcasc/jenkins.yaml
/opt/jenkins_robotframework/jenkins-integration/jobs/robot-execution-job.groovy
```

8.1 全局环境与凭据

当前推荐使用 JCasC 管理 Jenkins 全局环境变量，不再到 Jenkins UI 里手工维护。

`jenkins-integration/jcasc/jenkins.yaml` 中定义了 Robot 主线需要的全局环境：

```
ROBOTWS_REPO_URL=git@wrgitlab.ext.net.nokia.com:RAN/robotws.git
TESTLINE_CONFIGURATION_REPO_URL=git@wrgitlab.ext.net.nokia.com:RAN/configuration-
management/testline_configuration.git
PIP_INDEX_URL=${PIP_INDEX_URL}
PIP_EXTRA_INDEX_URL=${PIP_EXTRA_INDEX_URL}
PIP_TRUSTED_HOST=${PIP_TRUSTED_HOST}
```

`PIP_*` 属于敏感或环境私有值，仍通过 Jenkins controller 环境变量提供。Controller 只保存 `t813-agent` 启动用的 SSH launcher 私钥；`robotws` / `testline_configuration` 的 checkout 私钥改为保留在 agent 本机，通过 agent 环境变量提供 key path。参考文件：

```
deploy/env/jenkins-jcasc.env.example
```

关于 `T813_AGENT_SSH_PRIVATE_KEY`、`ROBOTWS_GIT_SSH_PRIVATE_KEY`、`TESTLINE_CONFIGURATION_GIT_SSH_PRIVATE_KEY` 的生成命令、公钥投放位置、JCasC env 填法与验证步骤，统一参考 c:/TA/jenkins_robotframework/docs/overview/jenkins-ssh-key-setup.md。

服务器上可以创建真实文件，例如：

```
/etc/default/jenkins-jcasc
```

内容按实际环境填写：

```
JENKINS_URL=https://10.71.210.104/jenkins/
JENKINS_ADMIN_EMAIL=admin@example.com

T813_AGENT_SSH_USER=jenkins
T813_AGENT_SSH_PRIVATE_KEY_PATH=/etc/jenkins/keys/jenkins_agent_rsa

ROBOTWS_GIT_SSH_KEY_PATH=/home/jenkins/.ssh/jenkins_gitlab_rsa
TESTLINE_CONFIGURATION_GIT_SSH_KEY_PATH=/home/jenkins/.ssh/jenkins_gitlab_rsa
```

```
PIP_INDEX_URL=https://<user>:<token>@artifactory-
hz1.ext.net.nokia.com/artifactory/api/pypi/ute-pypi-virtual/simple
PIP_EXTRA_INDEX_URL=https://<user>:<token>@artifactory-
espoo2.int.net.nokia.com/artifactory/api/pypi/ute-pypi-virtual/simple
PIP_TRUSTED_HOST=artifactory-hz1.ext.net.nokia.com artifactory-
espoo2.int.net.nokia.com
```

同时把 controller 连接 agent 的真实私钥单独放到 controller 本机文件，例如：

```
sudo install -d -m 0700 /etc/jenkins/keys
sudo install -o jenkins -g jenkins -m 0600 ~/.ssh/jenkins_agent_rsa
/etc/jenkins/keys/jenkins_agent_rsa
```

`jenkins_agent_rsa` 仍要求是 PEM RSA。

`robotws` / `testline_configuration` 的 GitLab key 不再复制到 controller。请确保这把 key 已经存在于 `t813-agent` 本机，并且上面两个 `*_GIT_SSH_KEY_PATH` 指向的文件路径对 Jenkins agent 进程可读。

把 JCasC 配置和 env 文件接入 Jenkins systemd override：

```
sudo systemctl edit jenkins
```

写入或合并：

```
[Service]
Environment="CASC_JENKINS_CONFIG=/var/lib/jenkins/casc_configs/jenkins.rendered.ya
ml"
EnvironmentFile=-/etc/default/jenkins-jcasc
ExecStartPre=/usr/bin/python3 /opt/jenkins_robotframework/jenkins-
integration/scripts/render_jcasc.py --template
/opt/jenkins_robotframework/jenkins-integration/jcasc/jenkins.yaml --output
/var/lib/jenkins/casc_configs/jenkins.rendered.yaml
```

应用：

```
sudo systemctl daemon-reload
sudo systemctl restart jenkins
```

如果 `ExecStartPre` 失败，先检查 controller 上的 launcher key path 是否存在，并且对 Jenkins 服务用户可读：

```
sudo -u jenkins test -r /etc/jenkins/keys/jenkins_agent_rsa
```

如果 job checkout 失败，再到 `t813-agent` 上检查：

```
test -r /home/jenkins/.ssh/jenkins_gitlab_rsa
```

如果 Jenkins 已安装 Configuration as Code 插件，也可以在页面执行 reload：

```
Manage Jenkins -> Configuration as Code -> Reload existing configuration
```

验证 Jenkins 全局变量是否由 JCasC 注入：

```
Manage Jenkins -> System -> Global properties -> Environment variables
```

也可以跑一次 `robot/robot-execution`，查看 checkout 阶段生成的：

```
artifacts/source-checkout.json
```

其中 `robotws` 和 `testline_configuration` 的 `repo_url` 不应再是 `null`。

如果暂时不使用 JCasC，才需要在 Jenkins 页面手工配置：

```
Manage Jenkins -> System -> Global properties -> Environment variables
Manage Jenkins -> Credentials -> System -> Global credentials
```

8.2 Robot Execution Job

目标 job 路径：

```
robot/robot-execution
```

Pipeline 文件：

```
jenkins-integration/pipelines/robot-execution.Jenkinsfile
```

Job DSL 文件：

```
jenkins-integration/jobs/robot-execution-job.groovy
```

当前 `platform-api` trigger 默认调用：

```
JENKINS_ROBOT_JOB_PATH=job/robot/job/robot-execution
```

注意：这里指的是 Jenkins UI 里的 job 路径，不是 Linux 服务器上的目录路径。

如果你在未登录状态下执行：

```
curl -I http://127.0.0.1:8080/jenkins/job/robot/job/robot-execution/
```

返回 `403 Forbidden`，只能说明 Jenkins 已启用鉴权，不能单靠这个结果判断 job 一定存在或一定不存在。更可靠的做法是登录 Jenkins 页面后确认，或者在 Jenkins 中直接手工创建这个 job。

8.2.1 最短手工创建 `robot/robot-execution` job

这部分详细步骤已经统一收口到 c:/TA/jenkins_robotframework/docs/overview/automation-portal-robot-case-flow.md。

这里仅保留部署角度最关键的结论：

1. 如果当前 Jenkins 里还没有 `robot/robot-execution`，第一轮可以直接在 Jenkins 页面手工创建，不必先走 Job DSL。
2. `Definition` 选择 `Pipeline script from SCM`。
3. `Repository URL` 默认使用：
   `https://github.com/stella555359/jenkins_robotframework.git`。
4. `Script Path` 固定为：`jenkins-integration/pipelines/robot-execution.Jenkinsfile`。
5. `Branch Specifier` 应填写 Jenkins 当前实际要读取的分支，而不是盲目沿用文档旧值。
6. 如果仓库访问需要凭据，再按 HTTPS 或 SSH 方式选择对应 Jenkins credentials。

需要以下内容时，直接看 overview 文档，不再在本节重复维护：

1. Jenkins 页面逐项怎么填。
2. HTTPS / SSH 两种 SCM 填法。
3. `Repository browser`、`Additional Behaviours`、`Lightweight checkout` 的建议。
4. `Build with Parameters` 为什么不出现。
5. Jenkins workspace 和 `/opt/jenkins_robotframework` 的区别。

8.2.2 创建后立即验证

建议创建后立刻做一次手工 smoke，确认 Jenkinsfile 能被正常读取、参数面出现、Agent 能接任务、checkout 和 Python 环境可用。

推荐的最小 smoke 参数、参数清单和排查方法也统一放在 c:/TA/jenkins_robotframework/docs/overview/automation-portal-robot-case-flow.md。

8.3 Jenkins Job 参数

`robot-execution` 的详细参数列表、Portal 字段到 Jenkins 参数的映射、以及 `PLATFORM_API_BASE_URL` 的作用，也统一在 c:/TA/jenkins_robotframework/docs/overview/automation-portal-robot-case-flow.md 维护。

本手册只保留两点部署侧必须确认的结论：

1. `PLATFORM_API_BASE_URL` 必须是外部 HTTPS 根地址，不带 `/api`，例如 `https://10.71.210.104`。
2. Jenkins callback 会基于它拼出 `/api/runs/{run_id}/callbacks/jenkins`，因此这个地址必须能从 Jenkins Master / Agent 访问。

9. platform-api 部署

当前服务器排查结果如果是：

```
systemctl status platform-api -> Unit platform-api.service could not be found
ss -ltnp | grep 8000 -> 无输出
curl http://127.0.0.1:8000/api/health -> Could not connect to server
```

说明不是 Nginx 配错，而是 `platform-api` 还没有部署完成。此时应先完成本节，再回头验证 `/api/health`。

9.1 代码目录

```
/opt/jenkins_robotframework/platform-api
```

9.2 Python venv 与依赖

```
cd /opt/jenkins_robotframework/platform-api
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
deactivate
```

如果 `venv` 还没创建过，可以先确认目录内容：

```
ls -la /opt/jenkins_robotframework/platform-api
```

9.2.1 先手工启动验证

在创建 systemd 之前，先手工启动一次，确认应用本身能跑起来：

```
cd /opt/jenkins_robotframework/platform-api
source venv/bin/activate
venv/bin/python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

另开一个终端验证：

```
curl http://127.0.0.1:8000/api/health
```

只有这一步通过，才继续创建 systemd。否则先看启动报错，不要跳过。

9.3 platform-api 环境变量

配置文件建议位置：

```
/opt/jenkins_robotframework/platform-api/.env
```

如果 `.env` 还不存在，可以先创建一个最小可用版本：

```
cat > /opt/jenkins_robotframework/platform-api/.env <<'EOF'
APP_NAME="Platform API"
APP_ENV=production
APP_HOST=127.0.0.1
APP_PORT=8000
RUNS_DB_PATH=/var/lib/test-workflow-runner/data/automation_platform.db

PUBLIC_BASE_URL=https://10.71.210.104

JENKINS_BASE_URL=http://127.0.0.1:8080/jenkins
JENKINS_ROBOT_JOB_PATH=job/robot/job/robot-execution
JENKINS_USERNAME=<jenkins-user>
JENKINS_API_TOKEN=<jenkins-api-token>
JENKINS_TRIGGER_TOKEN=
JENKINS_TIMEOUT_SECONDS=30
EOF
```

示例：

```
APP_NAME="Platform API"
APP_ENV=production
APP_HOST=127.0.0.1
APP_PORT=8000
RUNS_DB_PATH=/var/lib/test-workflow-runner/data/automation_platform.db

PUBLIC_BASE_URL=https://10.71.210.104
```

```
JENKINS_BASE_URL=http://127.0.0.1:8080/jenkins
JENKINS_ROBOT_JOB_PATH=job/robot/job/robot-execution
JENKINS_USERNAME=<jenkins-user>
JENKINS_API_TOKEN=<jenkins-api-token>
JENKINS_TRIGGER_TOKEN=
JENKINS_TIMEOUT_SECONDS=30
```

说明：

- `PUBLIC_BASE_URL` 是 Jenkins callback 使用的外部根地址，不带 `/api`。
- `JENKINS_BASE_URL` 可以走本机 Jenkins 地址，不需要走公网 HTTPS。
- `JENKINS_API_TOKEN` 从 Jenkins 用户页面生成，不要提交到 Git。

9.4 systemd 服务

系统文件位置：

```
/etc/systemd/system/platform-api.service
```

内容：

```
[Unit]
Description=Platform API Service
After=network.target

[Service]
Type=simple
User=ute
WorkingDirectory=/opt/jenkins_robotframework/platform-api
Environment="PATH=/opt/jenkins_robotframework/platform-api/venv/bin"
ExecStart=/opt/jenkins_robotframework/platform-api/venv/bin/uvicorn app.main:app -
-host 127.0.0.1 --port 8000
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

如果当前服务器还没有这个 service 文件，可直接创建：

```
sudo tee /etc/systemd/system/platform-api.service > /dev/null <<'EOF'
[Unit]
Description=Platform API Service
After=network.target

[Service]
```

```
Type=simple
User=ute
WorkingDirectory=/opt/jenkins_robotframework/platform-api
Environment="PATH=/opt/jenkins_robotframework/platform-api/venv/bin"
ExecStart=/opt/jenkins_robotframework/platform-api/venv/bin/uvicorn app.main:app -
-host 127.0.0.1 --port 8000
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
```

启动：

```
sudo systemctl daemon-reload
sudo systemctl enable platform-api
sudo systemctl restart platform-api
sudo systemctl status platform-api --no-pager
```

如果启动失败，立即看最近日志：

```
sudo journalctl -u platform-api -n 100 --no-pager
```

查看日志：

```
journalctl -u platform-api -f
```

本机验证：

```
curl http://127.0.0.1:8000/api/health
```

Nginx HTTPS 验证：

```
curl -k https://127.0.0.1/api/health
```

结果判读：

1. `curl http://127.0.0.1:8000/api/health` 成功，说明应用本身已启动。
2. `curl -k https://127.0.0.1/api/health` 成功，说明 Nginx 到 `platform-api` 的反代也已恢复。
3. 如果本机 `8000` 不通，就不要继续看 Nginx，先修 `platform-api` 本身。

9.5 AI analysis worker systemd 服务

`AI Run Insight` 点击生成后，后端只会把任务写入 `ai_analysis` 队列表。真正把 queued 任务推进到 `completed` / `failed` 的是独立 worker：

```
platform-api/app/services/ai_analysis_worker.py
```

当前生产默认使用 `rules_first`，不需要 `CURSOR_API_KEY`。只有显式请求 `analysis_mode=cursor_sdk` 时，才需要 Cursor Cloud Agents API 权限。

仓库内 service 模板：

```
/opt/jenkins_robotframework/deploy/systemd/platform-api-ai-worker.service
```

系统文件位置：

```
/etc/systemd/system/platform-api-ai-worker.service
```

安装：

```
sudo cp /opt/jenkins_robotframework/deploy/systemd/platform-api-ai-worker.service
\
  /etc/systemd/system/platform-api-ai-worker.service

sudo systemctl daemon-reload
sudo systemctl enable platform-api-ai-worker
sudo systemctl restart platform-api-ai-worker
sudo systemctl status platform-api-ai-worker --no-pager
```

查看日志：

```
sudo journalctl -u platform-api-ai-worker -n 100 --no-pager
sudo journalctl -u platform-api-ai-worker -f
```

服务通过标准：

```
1. systemctl status platform-api-ai-worker 显示 active (running)。
2. Portal 点击 Generate Rules Analysis 后，GET /ai-analysis 先看到 queued。
3. 几秒后 GET /ai-analysis 变为 completed 或 failed。
4. completed 时页面展示 Summary / RCA / Evidence / Report Preview。
```

常见失败判断：

```
一直 queued:
  platform-api-ai-worker 没启动，或 worker 没连到同一个 RUNS_DB_PATH。

worker 启动后立刻退出:
  查看 journalctl，优先确认 venv 路径、WorkingDirectory 和依赖安装。

completed 但 RCA category=unknown:
  rules_first 没命中当前内置规则，需要补规则或提供更完整 console/artifact evidence。

failed 且提示 Cursor / 403:
  说明请求走了 cursor_sdk，不是 rules_first。先确认 Portal 或 curl 请求体里的
analysis_mode。
```

10. automation-portal 部署

当前服务器排查结果如果是：

```
ls -la /opt/jenkins_robotframework/automation-portal/dist -> No such file or
directory
curl -k -I https://127.0.0.1/ -> 500 Internal Server Error
```

说明前端构建产物 `dist/` 还不存在，Nginx 虽然已经指向了这个目录，但目前没有可提供的静态页面。本节完成后，首页 `/` 的 500 才会消失。

10.1 代码目录

```
/opt/jenkins_robotframework/automation-portal
```

10.2 Node.js

前端构建需要 Node.js 20 LTS。可以在 Master、Jenkins 或专用构建机上安装。服务器最终只需要部署 `dist/`。

验证：

```
node --version
npm --version
```

预期：

```
node v20.x.x
```

```
npm 可用
```

如果 `node` 或 `npm` 不存在，先补齐 Node.js 20 LTS，再继续本节。

10.3 前端环境变量

配置文件位置：

```
/opt/jenkins_robotframework/automation-portal/.env
```

内容：

```
VITE_APP_TITLE=Automation Portal
VITE_API_BASE_URL=/api
VITE_JENKINS_BASE_URL=https://10.71.210.104/jenkins
```

说明：

- `VITE_API_BASE_URL` 必须使用 `/api` 相对路径，这样 Windows 浏览器访问 `https://10.71.210.104/` 时会同域调用 `https://10.71.210.104/api/...`。
- `VITE_JENKINS_BASE_URL` 是 Portal 页面上 Jenkins 链接的目标地址。Portal 会根据 `jenkins_build_ref`（格式 `robot/robot-execution#42`）自动拼出 Jenkins build 页面 URL 和 archived artifact URL。如果不配或为空，Portal 的 Jenkins 链接和 artifact 下载按钮不会显示。

如果 `.env` 还不存在，可先创建：

```
cat > /opt/jenkins_robotframework/automation-portal/.env <<'EOF'
VITE_APP_TITLE="Automation Portal"
VITE_API_BASE_URL=/api
VITE_JENKINS_BASE_URL=https://10.71.210.104/jenkins
EOF
```

10.4 构建

```
cd /opt/jenkins_robotframework/automation-portal
npm install
npm run build
```

当前项目 `package.json` 的构建脚本是：

```
tsc -b && vite build
```

所以如果构建失败，既可能是前端依赖没装好，也可能是 TypeScript 编译没通过。

10.4.1 服务器本地忽略构建副产物

在服务器上执行 `npm install`、`npm run build` 后，可能会看到类似这些本地文件出现在 `git status`：

```
automation-portal/node_modules/
automation-portal/package-lock.json
automation-portal/src/*.js
automation-portal/src/*.d.ts
automation-portal/*.tsbuildinfo
automation-portal/vite.config.js
automation-portal/vite.config.d.ts
```

这些通常是服务器本地安装和构建产生的副产物，不代表必须 push 回 GitHub。若只是想让这台服务器本地不再显示这些未跟踪文件，推荐写入：

```
/opt/jenkins_robotframework/.git/info/exclude
```

这个文件只对当前服务器生效，不会进入 Git 提交。

执行：

```
cd /opt/jenkins_robotframework
cat >> .git/info/exclude <<'EOF'
automation-portal/node_modules/
automation-portal/package-lock.json
automation-portal/src/**/*.js
automation-portal/src/**/*.d.ts
automation-portal/vite.config.js
automation-portal/vite.config.d.ts
automation-portal/*.tsbuildinfo
EOF
```

验证：

```
git status
git status --ignored
```

说明：

1. 这是服务器本地忽略规则，不会影响其他开发机。
2. 它只对当前还未被 Git 跟踪的文件有效。
3. 如果团队后续决定正式提交 `package-lock.json`，再把对应这一行从 `.git/info/exclude` 删除即可。

构建产物：

```
/opt/jenkins_robotframework/automation-portal/dist
```

构建后建议立刻确认首页文件是否存在：

```
ls -la /opt/jenkins_robotframework/automation-portal/dist
ls -la /opt/jenkins_robotframework/automation-portal/dist/index.html
```

Nginx 的 `/` 已经配置为：

```
location / {
    root /opt/jenkins_robotframework/automation-portal/dist;
    try_files $uri $uri/ /index.html;
}
```

验证：

```
ls -la /opt/jenkins_robotframework/automation-portal/dist
curl -k -I https://127.0.0.1/
```

如果 `dist/` 已存在但首页仍然返回 `500`，继续检查：

```
namei -l /opt/jenkins_robotframework/automation-portal/dist/index.html
sudo journalctl -u nginx -n 100 --no-pager
```

重点看是否是目录权限或文件权限导致 Nginx 读不到 `index.html`。

Windows 浏览器打开：

```
https://10.71.210.104/
```

11. test-workflow-runner 模块定位

11.1 代码目录

```
/opt/jenkins_robotframework/test-workflow-runner
```

11.2 当前角色

`test-workflow-runner` 不是 Portal、platform-api、Jenkins Pipeline 或 Robot case 公共调度层。

它的当前定位是：

```
一个 Python orchestrator 执行器
```

它负责：

1. 在执行侧读取 workflow JSON。
2. 加载 `env_map.json` 和 testline configuration。
3. 编排并执行 attach、handover、dl_traffic、ul_traffic、swap、detach、syslog_check 等 stage handler。
4. 执行 `kpi_generator`、`kpi_detector` 这类 follow-up / internal tool 能力。
5. 产出执行结果 JSON，并作为 runner CLI 被上层集成调用。
6. 提供 standalone internal tool 执行入口，例如 `internal_tools.tool_runner` 和 `internal_tools.worker`。

它不负责：

1. Portal 页面与 API 接口。
2. Jenkins 公共 Pipeline 编排。
3. 通用 checkout、workspace bootstrap、callback 回写。
4. Robot case 主线的公共桥接逻辑。

可以把它理解为：

```
与 Robot case 公共链路无关
但与 standalone internal tool 能力直接相关
```

这些公共调度和桥接逻辑当前在 `jenkins-integration/`，而 `test-workflow-runner` 更接近“被调用的执行器”以及“standalone internal tool 的执行宿主”，而不是“整个链路本身”。

11.3 上层集成调用时的运行环境

当 `test-workflow-runner` 由 Jenkins integration 从 Agent 侧调用时，建议运行环境为：

```
/automation/workspace
```

这类调用通常还依赖：

```
robotws
testline_configuration
Python environment, for example /home/ute/CIENV/<TESTLINE>
```

具体的 checkout、命令生成和桥接仍由这些脚本处理：

```
jenkins-integration/scripts/materialize_run_request.py
jenkins-integration/scripts/checkout_sources.py
jenkins-integration/scripts/prepare_taf_environment.py
jenkins-integration/scripts/build_robot_command.py
jenkins-integration/scripts/post_run_callback.py
```

12. standalone internal_tools worker 部署

如果只跑 Robot case，worker 可以先不启用。
如果 Portal 后续要独立触发 `kpi_generator` / `kpi_detector`，需要启用 worker。

12.1 代码目录

```
/opt/jenkins_robotframework/test-workflow-runner
```

12.2 Python venv

当前 `test-workflow-runner` 已补充独立 `requirements.txt`，用于安装 standalone internal tool 与 `kpi_generator` / `kpi_detector` 相关运行依赖。

```
cd /opt/jenkins_robotframework/test-workflow-runner
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
deactivate
```

当前文件已覆盖仓库内明确使用到的运行时第三方依赖：`numpy`、`pandas`、`scipy`、`openpyxl`、`requests`、`urllib3`。如果后续新增依赖，再同步更新该文件。

12.3 worker systemd 服务

系统文件位置：

```
/etc/systemd/system/internal-tools-worker.service
```

如果当前服务器还没有这个 service 文件，可直接创建：

```
sudo tee /etc/systemd/system/internal-tools-worker.service > /dev/null <<'EOF'
[Unit]
Description=Internal KPI Tools Worker
```

```
After=network.target platform-api.service
Requires=platform-api.service

[Service]
Type=simple
User=ute
WorkingDirectory=/opt/jenkins_robotframework/test-workflow-runner
Environment="PATH=/opt/jenkins_robotframework/test-workflow-runner/venv/bin"
ExecStart=/opt/jenkins_robotframework/test-workflow-runner/venv/bin/python -m
internal_tools.worker --platform-api-base-url http://127.0.0.1:8000 --output-root
/var/lib/test-workflow-runner/internal-tools/artifacts --poll-interval-seconds 10
--limit 1
Restart=always
RestartSec=5
StandardOutput=append:/var/lib/test-workflow-runner/internal-tools/logs/worker.log
StandardError=append:/var/lib/test-workflow-runner/internal-
tools/logs/worker.err.log

[Install]
WantedBy=multi-user.target
EOF
```

内容：

```
[Unit]
Description=Internal KPI Tools Worker
After=network.target platform-api.service
Requires=platform-api.service

[Service]
Type=simple
User=ute
WorkingDirectory=/opt/jenkins_robotframework/test-workflow-runner
Environment="PATH=/opt/jenkins_robotframework/test-workflow-runner/venv/bin"
ExecStart=/opt/jenkins_robotframework/test-workflow-runner/venv/bin/python -m
internal_tools.worker --platform-api-base-url http://127.0.0.1:8000 --output-root
/var/lib/test-workflow-runner/internal-tools/artifacts --poll-interval-seconds 10
--limit 1
Restart=always
RestartSec=5
StandardOutput=append:/var/lib/test-workflow-runner/internal-tools/logs/worker.log
StandardError=append:/var/lib/test-workflow-runner/internal-
tools/logs/worker.err.log

[Install]
WantedBy=multi-user.target
```

启动：

```
sudo systemctl daemon-reload
sudo systemctl enable internal-tools-worker
sudo systemctl restart internal-tools-worker
sudo systemctl status internal-tools-worker --no-pager
```

日志：

```
journalctl -u internal-tools-worker -f
tail -f /var/lib/test-workflow-runner/internal-tools/logs/worker.log
tail -f /var/lib/test-workflow-runner/internal-tools/logs/worker.err.log
```

单次手工验证：

```
cd /opt/jenkins_robotframework/test-workflow-runner
source venv/bin/activate
python -m internal_tools.worker \
  --platform-api-base-url http://127.0.0.1:8000 \
  --output-root /var/lib/test-workflow-runner/internal-tools/artifacts \
  --once
```

13. 防火墙与端口策略

对 Windows 用户只开放 HTTPS：

```
443/tcp -> Nginx
80/tcp  -> Nginx redirect 到 HTTPS，可选
```

不建议对外开放：

```
8000/tcp -> platform-api 仅监听 127.0.0.1
8080/tcp -> Jenkins 由 Nginx /jenkins/ 代理
5173/tcp -> Vite dev server 不用于生产部署
```

如果启用了防火墙：

```
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw status
```

14. 端到端验证

14.1 服务状态

```
sudo systemctl status jenkins --no-pager
sudo systemctl status platform-api --no-pager
sudo systemctl status nginx --no-pager
```

如果启用 worker：

```
sudo systemctl status internal-tools-worker --no-pager
```

14.2 本机 curl

```
curl -k -I https://127.0.0.1/
curl -k https://127.0.0.1/api/health
curl -k -I https://127.0.0.1/jenkins/
```

14.3 Windows 浏览器

```
https://10.71.210.104/
https://10.71.210.104/api/health
https://10.71.210.104/jenkins/
```

自签名证书会有浏览器安全提示，第一轮可以手工继续访问；正式环境应换成公司信任证书。

14.4 Robot run API smoke

创建 run：

```
curl -k -X POST https://127.0.0.1/api/runs \
  -H "Content-Type: application/json" \
  -d '{
    "testline": "T813",
    "robotcase_path": "testsuite/Hangzhou/RRM/example.robot",
    "executor_type": "robot",
    "metadata": {
      "case_name": "Attach Smoke",
      "selected_tests": ["Attach UE"],
      "robot_variables": {}
    }
  }'
```

拿到 `run_id` 后触发：

```
curl -k -X POST https://127.0.0.1/api/runs/<run_id>/trigger
```

查看详情：

```
curl -k https://127.0.0.1/api/runs/<run_id>
```

14.5 Portal smoke

Windows 打开：

```
https://10.71.210.104/
```

页面操作：

1. 进入 `New Robot Run`。
2. 填 `testline`。
3. 填 `robotcase_path`。
4. 可选填 `case_name`、`selected_tests`、`robot_variables`。
5. 点击 `Run`。
6. 自动跳转到 `/runs/<run_id>`。
7. Jenkins 执行结束后，详情页应看到 `passed` 或 `failed`，以及 `jenkins_build_ref` 和 artifacts。

15. 常见问题排查

15.1 Windows 能打开 Jenkins，但打不开 Portal

检查：

```
ls -la /opt/jenkins_robotframework/automation-portal/dist
sudo nginx -t
sudo journalctl -u nginx --since "10 minutes ago"
```

如果 `dist/` 不存在，重新构建：

```
cd /opt/jenkins_robotframework/automation-portal
npm install
npm run build
sudo systemctl reload nginx
```

15.2 Portal 能打开，但 API 失败

检查：

```
sudo systemctl status platform-api --no-pager
journalctl -u platform-api -n 100 --no-pager
curl http://127.0.0.1:8000/api/health
curl -k https://127.0.0.1/api/health
```

确认 Nginx 里有：

```
location /api/ {
    proxy_pass http://127.0.0.1:8000/api/;
}
```

15.3 Trigger Jenkins 失败

检查 `platform-api/.env`：

```
JENKINS_BASE_URL=http://127.0.0.1:8080/jenkins
JENKINS_ROBOT_JOB_PATH=job/robot/job/robot-execution
JENKINS_USERNAME=<jenkins-user>
JENKINS_API_TOKEN=<jenkins-api-token>
```

检查 Jenkins job 是否存在：

```
https://10.71.210.104/jenkins/job/robot/job/robot-execution/
```

查看 API 日志：

```
journalctl -u platform-api -f
```

15.4 Jenkins 执行完没有回写状态

检查 Jenkins job 参数：

```
PLATFORM_API_BASE_URL=https://10.71.210.104
CALLBACK_INSECURE_TLS=true
```

检查 `platform-api` 到 Jenkins 的触发配置：

```
JENKINS_BASE_URL=http://127.0.0.1:8080/jenkins
JENKINS_INSECURE_TLS=false
```

如果你把 `JENKINS_BASE_URL` 配成了外部 `https://10.71.210.104/jenkins`，且当前还是自签名证书，就需要额外设置：

```
JENKINS_INSECURE_TLS=true
```

检查 callback URL 是否可从 Jenkins Master/Agent 访问：

```
curl -k https://10.71.210.104/api/health
```

检查 Jenkins 构建日志里 `post_run_callback.py` 的输出。如果看到 `CERTIFICATE_VERIFY_FAILED`，说明当前还是自签名证书场景，但 callback 没有跳过 TLS 校验。

15.5 Agent 连不上

从 Master 验证：

```
ssh -i ~/.ssh/jenkins_agent_rsa jenkins@10.57.159.149 "java -version && pwd"
```

检查 Agent：

```
systemctl status ssh --no-pager || systemctl status sshd --no-pager
ls -ld /automation/workspace
```

15.6 internal_tools worker 不消费任务

检查：

```
curl http://127.0.0.1:8000/api/kpi/tool-runs?status=created
sudo systemctl status internal-tools-worker --no-pager
journalctl -u internal-tools-worker -n 100 --no-pager
```

确认 worker 参数：

```
--platform-api-base-url http://127.0.0.1:8000
--output-root /var/lib/test-workflow-runner/internal-tools/artifacts
```

16. 配置文件总表

模块	仓库位置	服务器实际位置
Nginx HTTPS 反代	`deploy/nginx/jenkins-kpi-platform.conf`	`/etc/nginx/sites-available/jenkins-kpi-platform.conf`
platform-api env	`platform-api/.env` 不提交真实密钥	`/opt/jenkins_robotframework/platform-api/.env`
platform-api systemd	建议后续沉淀到 `deploy/systemd/platform-api.service`	`/etc/systemd/system/platform-api.service`
Portal env	`automation-portal/.env` 不提交环境私有值	`/opt/jenkins_robotframework/automation-portal/.env`，含 `VITE_JENKINS_BASE_URL`
Portal dist	`automation-portal/dist` 构建产物	`/opt/jenkins_robotframework/automation-portal/dist`
Jenkinsfile	`jenkins-integration/pipelines/robot-execution.Jenkinsfile`	Jenkins job 从仓库读取
Jenkins JCasC	`jenkins-integration/jcasc/jenkins.yaml`	Jenkins Configuration as Code 使用
Jenkins Job DSL	`jenkins-integration/jobs/robot-execution-job.groovy`	Seed job 或 Job DSL 使用
internal_tools worker systemd	建议后续沉淀到 `deploy/systemd/internal-tools-worker.service`	`/etc/systemd/system/internal-tools-worker.service`
internal_tools artifacts	无需提交	`/var/lib/test-workflow-runner/internal-tools/artifacts`
SQLite DB	无需提交	`/var/lib/test-workflow-runner/data/automation_platform.db`

17. 一句话验收标准

部署完成后，Windows 只需要访问：

```
https://10.71.210.104/
```

然后在 Portal 上创建 Robot run。后端应自动：

```
Portal -> /api/runs -> /api/runs/{run_id}/trigger -> Jenkins robot/robot-execution
-> Robot -> /api/runs/{run_id}/callbacks/jenkins -> Portal detail
```

整个过程中，用户不需要直接访问 `:5173`、`:8000`、`:8080`。

18. 服务器代码更新后操作集合

这一节针对你已经把代码改好并推到仓库后，服务器侧为了让最新变更真正生效，需要执行哪些动作。

18.1 先更新服务器部署副本

```
cd /opt/jenkins_robotframework
git fetch --all --prune
git pull --ff-only
```

如果你当前不是部署分支，先显式切到目标分支再拉：

```
cd /opt/jenkins_robotframework
git checkout <your-branch>
git pull --ff-only
```

18.2 platform-api 代码改动后

只要改到了 `platform-api/` 下的 Python 代码，例如这次的 Jenkins 参数透传逻辑，就执行：

```
sudo systemctl restart platform-api
sudo systemctl status platform-api --no-pager
journalctl -u platform-api -n 100 --no-pager
```

18.3 automation-portal 前端代码改动后

只要改到了 `automation-portal/` 下的前端页面或静态资源，例如这次新增 `TAF mode` 和 `Robotws git ref` 字段，就执行：

```
cd /opt/jenkins_robotframework/automation-portal
npm run build
sudo systemctl reload nginx
```

如果这台服务器上还没装过前端依赖，或者 `package.json` / `package-lock.json` 有变化，再先执行一次：

```
cd /opt/jenkins_robotframework/automation-portal
npm install
npm run build
sudo systemctl reload nginx
```

18.4 Jenkins pipeline / 脚本改动后

只要改到了下面这些文件：

```
jenkins-integration/pipelines/robot-execution.Jenkinsfile
jenkins-integration/scripts/*.py
```

通常不需要重启 Jenkins 服务。下一次 `robot/robot-execution` 从 SCM 读取最新仓库内容时就会生效。

但要注意两点：

1. 如果 job 是 `Pipeline script from SCM`，先确认 Jenkins job 的仓库分支已经指到包含最新提交的分支。
2. 如果你还在用手工创建的 job，而不是 Seed Job / Job DSL，新增参数时可能需要在 Jenkins 页面重新保存一次 job 配置，或者先手工跑一次，让 Declarative Pipeline 把参数定义刷新出来。

如需确认 Jenkinsfile 已更新到最新提交，可直接在 Jenkins 再触发一次 `Build with Parameters`，检查是否已经出现例如：

```
TAF_MODE
ROBOTWS_GIT_REF
```

18.5 Job DSL / JCasC 文件改动后

如果改的是：

```
jenkins-integration/jobs/robot-execution-job.groovy
jenkins-integration/jcasc/jenkins.yaml
```

那不是 `git pull` 完就自动生效。

你还需要按实际使用方式做额外动作：

1. 如果你使用 Seed Job 管理 Jenkins job，就重新运行一次 Seed Job。
2. 如果你使用 JCasC 管理 Jenkins，就执行 JCasC reload，或者重启 Jenkins 让配置重载。
3. 如果当前 Jenkins job 是手工维护的，就到 Jenkins UI 手工补齐新增参数，不要指望 `.groovy` 或 `jenkins.yaml` 自动生效。

18.6 Portal 前端改版部署步骤

本次 Portal 前端改动包括：

1. 整体背景改成浅蓝色，字体切换到 Calibri。
2. 从顶部导航栏改为左侧侧边栏布局，预留 KPI Generator / Anomaly Detector / Workflow Designer 入口。
3. Run Detail 展示完整 artifact 列表和可直接打开的 `log.html` 链接。
4. Run List 和 Run Detail 新增 Jenkins build 跳转链接。
5. Run List 和 Run Detail 新增 Rebuild 按钮，可从已有 run 预填充表单快速重跑。
6. `materialize_run_request.py` 新增 `--insecure-skip-tls-verify` 支持自签名 HTTPS 部署。

服务器侧要做的三步：

第一步：配置 Portal 前端环境变量

在 `/opt/jenkins_robotframework/automation-portal/.env` 中确认有以下内容：

```
VITE_APP_TITLE="Automation Portal"
VITE_API_BASE_URL=/api
VITE_JENKINS_BASE_URL=https://10.71.210.104/jenkins
```

如果文件不存在，新建：

```
cat > /opt/jenkins_robotframework/automation-portal/.env <<'EOF'
VITE_APP_TITLE="Automation Portal"
VITE_API_BASE_URL=/api
VITE_JENKINS_BASE_URL=https://10.71.210.104/jenkins
EOF
```

关键说明：

- `VITE_JENKINS_BASE_URL` 用于 Portal 页面拼接 Jenkins build 页面链接和 artifact 下载 URL。
- 它的值应与 Nginx 反代 Jenkins 的外部地址一致。
- 如果后续域名或端口变化，只需改这一个变量然后重新 `npm run build`。
- **不要**在这里填 `http://127.0.0.1:8080/jenkins`，因为这是 Portal 页面在浏览器端用的，必须是 Windows 浏览器能访问到的外部地址。

第二步：重新构建前端

```
cd /opt/jenkins_robotframework
git fetch --all --prune
git pull --ff-only

cd /opt/jenkins_robotframework/automation-portal
npm install
npm run build
```

说明：

- `npm install` 在 `package.json` 没变化时可以跳过，但如果不确定就执行一次，耗时不长。
- `npm run build` 会执行 `tsc -b && vite build`，产出 `dist/` 目录。
- Vite 只在 **构建时** 把 `.env` 里的 `VITE_*` 变量注入到 JS bundle 里，运行时不会再读 `.env`。所以如果改了 `.env`，必须重新 build。

构建完成后确认：

```
ls -la /opt/jenkins_robotframework/automation-portal/dist/index.html
```

第三步：部署到 Nginx 并验证

Nginx 配置不需要改，当前已经有：

```
location / {
    root /opt/jenkins_robotframework/automation-portal/dist;
    try_files $uri $uri/ /index.html;
}
```

直接 reload 让 Nginx 重新读取静态文件：

```
sudo systemctl reload nginx
```

本机验证：

```
curl -k -I https://127.0.0.1/
```

预期返回 `200 OK`。

Windows 浏览器验证：

```
https://10.71.210.104/
```

验证要点：

1. 左侧应出现深蓝色侧边栏，包含 Robot Execution / KPI Tools / Test Workflow 三个分区。
2. 整体背景应为浅蓝色。
3. 进入 Run List，每行应有 Rebuild 按钮；如果有已完成的 run，Jenkins 列应显示可点击的链接。
4. 进入某个已完成的 Run Detail：
    - Jenkins Build 行应显示可点击的 Jenkins build 链接。
    - 如果有 `log.html` artifact，应在 summary 区域显示 "Open log.html" 快速入口。

    - Artifacts 区域应列出所有 artifact，每个带 Open/Download 按钮。
    - Rebuild 按钮应跳转到 New Robot Run 表单并自动预填上次的参数。
5. 如果 Jenkins 链接或 artifact 下载链接不可用，检查 `.env` 中 `VITE_JENKINS_BASE_URL` 是否配置正确，以及是否重新执行了 `npm run build`。

附：Jenkins 侧同步确认

本次同时更新了 `materialize_run_request.py` 和 `robot-execution.Jenkinsfile`。Jenkins 侧不需要手工操作，只要 job 的 SCM 分支指向了包含本次提交的分支，下次构建时会自动读到最新代码。

可以确认的方式是手工或从 Portal 触发一次 `robot/robot-execution`，在 Console Output 里搜索：

```
--insecure-skip-tls-verify
```

如果 Materialize Run Request 阶段的命令里已经包含这个参数，说明 Jenkins 已经读到了最新的 Jenkinsfile 和脚本。

18.7 `create-venv` 的内部 Artifactory 配置

如果你启用 `TAF_MODE=create-venv`，并希望 Jenkins 在新建 CIENV 后自动安装 `robotws/dependencies.py<major><minor>-rf50.lock`，现在 `prepare_taf_environment.py` 会在自动安装分支里生成接近下面这种命令：

```
python -m pip install -r /automation/workspace/workspace/robot/robot-
execution/robotws/dependencies.py311-rf50.lock \
  --no-deps \
  -i https://artifactory-espoo2.int.net.nokia.com/artifactory/api/pypi/ute-pypi-
virtual/simple \
  --proxy http://10.158.100.9:8080 \
  --trusted-host artifactory-espoo2.int.net.nokia.com
```

不要再依赖 Agent 宿主机的 `pip.conf`。当前推荐直接在 Jenkins 配下面这几个环境变量：

```
PIP_INDEX_URL
PIP_EXTRA_INDEX_URL
PIP_TRUSTED_HOST
```

示例：

```
PIP_INDEX_URL=https://<user>:<token>@artifactory-
hz1.ext.net.nokia.com/artifactory/api/pypi/ute-pypi-virtual/simple
PIP_EXTRA_INDEX_URL=https://<user>:<token>@artifactory-
espoo2.int.net.nokia.com/artifactory/api/pypi/ute-pypi-virtual/simple
```

```
PIP_TRUSTED_HOST=artifactory-hz1.ext.net.nokia.com artifactory-
espoo2.int.net.nokia.com
```

如果你不想配成全局环境，也可以在 `robot/robot-execution` 的 `Build with Parameters` 里临时填写：

```
PIP_INDEX_URL_OVERRIDE
PIP_EXTRA_INDEX_URL_OVERRIDE
PIP_TRUSTED_HOST_OVERRIDE
```

生效顺序是：

```
PIP_INDEX_URL_OVERRIDE > PIP_INDEX_URL > PIP_EXTRA_INDEX_URL_OVERRIDE >
PIP_EXTRA_INDEX_URL
```

另外注意：

1. lock 文件安装会带 `--no-deps`。
2. 当前脚本固定使用 `--proxy http://10.158.100.9:8080`。
3. `PIP_TRUSTED_HOST` 如果配多个 host，会展开成多个 `--trusted-host`。

推荐优先用 Jenkins 全局环境，原因是：

1. 不需要每次手工填。
2. 不需要把内部 Artifactory 地址和凭据暴露给 Portal。
3. `prepare_taf_environment.py` 会优先使用 `*_OVERRIDE`，否则回退到 Jenkins 全局环境 `PIP_*`。